

PERSONAL PROJECT PORTFOLIO

Enterprise-Grade SIEM Implementation, Threat Detection & Security Analytics

Toochukwu Praise Ajoku

MSc Cyber Security | Keele University

Student Member, CII Sec

May 2026

Technology Stack	Elastic Stack v8.11 (Elasticsearch, Kibana, Fleet, Elastic Agent)
Target Environment	Ubuntu 24.04 LTS endpoint monitored via Fleet-managed Elastic Agent
Attack Simulation	Hydra SSH brute-force from Kali Linux (MITRE ATT&CK T1110)
Key Deliverables	SIEM deployment, detection rule engineering, dual-audience dashboards
Frameworks Applied	MITRE ATT&CK, Elastic Common Schema (ECS), UK GDPR / NCSC guidance
Skills Demonstrated	Log ingestion, field parsing, KQL, threshold alerting, SOC & executive dashboards

Executive Summary

This portfolio documents a self-directed technical project in which I designed, deployed, and validated a production-grade Security Information and Event Management (SIEM) environment using the Elastic Stack (version 8.11). The project covered the full security operations lifecycle — from infrastructure provisioning and agent deployment through adversarial simulation, detection engineering, and dashboard design for both technical and executive audiences. This design aligns with the "Security by Design" philosophy articulated by Melone (2021), ensuring the monitoring architecture is maintainable, verifiable, and capable of evolving with the threat landscape.

The environment was built on a virtual lab comprising an Ubuntu 24.04 LTS target server managed through Elastic Fleet, with adversarial telemetry generated by a Hydra SSH brute-force attack launched from a Kali Linux node. This simulated MITRE ATT&CK Technique T1110 (Brute Force) against the authentication layer of the target — the most common point of exploitation according to Stallings and Brown (2018) — generating the log data used to validate every component of the detection pipeline.

The project produced three principal outputs: a fully operational SIEM ingesting multi-source telemetry mapped to the Elastic Common Schema (ECS); a threshold-based detection rule using KQL capable of attributing automated attacks to specific source IPs; and two role-differentiated dashboards — one designed for SOC analysts requiring high-fidelity operational data, and one for non-technical management stakeholders requiring strategic risk visibility, reflecting the human-centred design principles of Sasse and Flechais (2005).

The critical reflection section addresses the trade-offs inherent in detection threshold design, the limitations of static-IP attribution logic against distributed adversaries, and the UK GDPR and workplace governance considerations that any production deployment would require.

Part 1: SIEM Deployment & Data Ingestion

1.1 Project Rationale & Architecture Overview

Modern security operations demand centralised telemetry aggregation. Distributed infrastructure generates log data at volumes and speeds that make manual review operationally impossible; a SIEM provides the ingestion, normalisation, and analytical layer that transforms raw log streams into actionable security intelligence. As Stallings and Brown (2018) assert, the ability to correlate events across multiple data streams is a fundamental requirement of effective security operations.

For this project I selected the Elastic Stack for three reasons: it is widely deployed in UK enterprise environments and features in NCSC (2022) guidance on security monitoring; its Fleet management architecture directly mirrors the centralised agent orchestration model used in large security operations centres; and its native support for the Elastic Common Schema (ECS) provided a structured data normalisation target that I could validate programmatically. This design aligns with Melone's (2021) principle that a monitoring system must be maintainable and verifiable across its full operational lifecycle.

The architecture I deployed followed a hub-and-spoke model: a central Elasticsearch and Kibana instance formed the data lake and analytical interface, with Elastic Fleet acting as the policy control plane. A single Elastic Agent enrolled into the Fleet policy on the Ubuntu target, streaming log data back to the central cluster over an encrypted channel. As Melone (2021) emphasises, the use of a single control plane for policy

orchestration ensures scalability — security policies can be propagated to thousands of agents simultaneously from a single management interface.

1.2 Installation & Environment Provisioning

1.2.1 Software Procurement and Package Installation

The Elastic Debian packages were downloaded directly from the official Elastic repository via Firefox and stored locally, ensuring supply-chain integrity. Local installation from verified packages rather than unattended apt sources allows the package hash to be verified before execution — a practical application of the Security by Design principle advocated by Melone (2021), which holds that the integrity of the software supply chain must be assured at every stage of the installation lifecycle.

- Navigate to the downloads directory and install core components:

```
cd ~/Downloads
sudo dpkg -i elasticsearch-8.11.0-amd64.deb
sudo dpkg -i kibana-8.11.0-amd64.deb
```

1.2.2 Service Initialisation & Cluster Health Verification

After package installation, I initialised the Elasticsearch service and validated cluster readiness via the REST health API before enrolling any agents. This health-check gate is an important operational discipline: enrolling agents before the cluster is stable can corrupt index mappings. As Melone (2021) notes, a structured installation lifecycle that ensures reproducibility is a key requirement for institutional security assurance.

- Elasticsearch service initialisation:

```
sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service
sudo systemctl start elasticsearch.service
```

- Generate a Kibana enrollment token to create a cryptographically authenticated link between the two services:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
```

- Initialise Kibana:

```
sudo systemctl enable kibana.service
sudo systemctl start kibana.service
```

- Validate cluster health before agent enrollment:

```
curl -u elastic:[password] -X GET https://localhost:9200/_cluster/health?pretty
```

Figure 1 below shows the Elastic Stack fully operational following successful installation and cluster health validation.

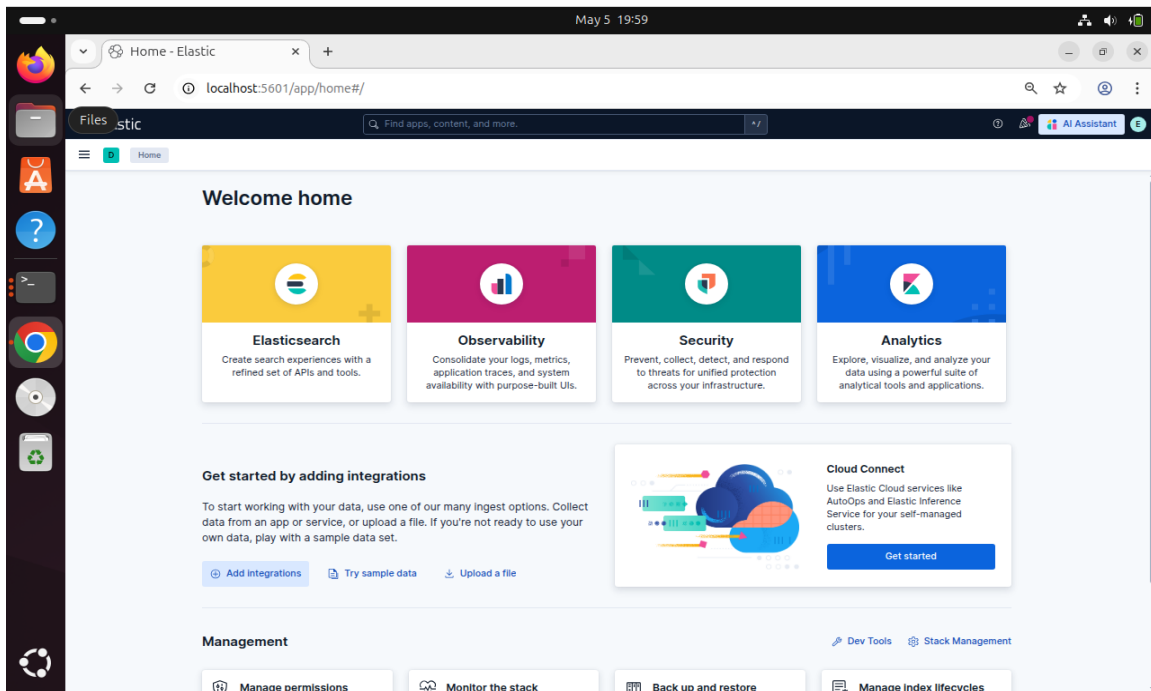


Figure 1: Elastic Stack (SIEM) installed and operational — Kibana dashboard accessible and cluster health confirmed green

1.3 Fleet-Based Agent Management

Rather than configuring individual log shippers on each endpoint, I used Elastic Fleet to define a centrally-managed agent policy. This is the approach used in enterprise SOC environments where a single policy change must propagate to hundreds of agents simultaneously. Fleet also provides real-time agent health visibility, allowing the security team to detect if an endpoint has stopped reporting — itself a potential indicator of compromise.

Figure 2 shows the Elastic Agent installation completing successfully on the Ubuntu target. The terminal output confirms the full enrollment sequence: TLS certificate generation, daemon restart, enrollment to the Fleet Server at localhost:8221, and final service confirmation — "Elastic Agent has been successfully installed."

```
[====] Done [1s]
Elastic Agent has been uninstalled.
chizzi@ubuntu-24: ~/Downloads/elastic-agent-9.3.4-linux-x86_64$ sudo ./elastic-agent install \
--fleet-server-es=https://localhost:9200 \
--fleet-server-service-token=AEEAAWsvYXN0aWwZmxlZlZlY2VydWVlZ3RvaZVUeTE3NzgwNTM1ODc2Mjg6TU9ran5wcdUlcFQmNkU081RDhwQQ \
--fleet-server-policy=fleet-server-policy \
--fleet-server-es-ca-trusted-fingerprint=793905feaa8aef75f783870fb4b07d916dec91499a8142912a632213326d5a \
--fleet-server-port=8220 \
--install-servers
Elastic Agent will be installed at /opt/Elastic/Agent and will run as a service. Do you want to continue? [Y/n]:y
[ ==] Service Started [7s] Elastic Agent successfully installed, starting enrollment.
[ ==] Waiting For Enroll... [9s] {"log.level":"info","@timestamp":"2026-05-06T07:49:31.532Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCnd).prepareFleetTLS","file.name":"cmd/enroll_cnd.go","file.line":351},"message":"Generating self-signed certificate for Fleet Server","ecs.version":"1.6.0"}
[ ==] Waiting For Enroll... [11s] {"log.level":"info","@timestamp":"2026-05-06T07:49:33.091Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCnd).daemonReloadWithBackoff","file.name":"cmd/enroll_cnd.go","file.line":395},"message":"Restarting agent daemon, attempt 0","ecs.version":"1.6.0"}
[ ==] Waiting For Enroll... [13s] {"log.level":"info","@timestamp":"2026-05-06T07:49:35.184Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.waitForFleetServer.func1","file.name":"cmd/enroll_cnd.go","file.line":572},"message":"Fleet Server - Running on policy with Fleet Server integration: fleet-server-policy; missing config fleet.t.agent.id (expected during bootstrap process)","ecs.version":"1.6.0"}
[ ==] Waiting For Enroll... [13s] {"log.level":"info","@timestamp":"2026-05-06T07:49:35.511Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/application/enroll.EnrollWithBackoff","file.name":"enroll/enroll.go","file.line":86},"message":"Starting enrollment to URL: https://localhost:8221/","ecs.version":"1.6.0"}
[ ==] Waiting For Enroll... [15s] {"log.level":"info","@timestamp":"2026-05-06T07:49:37.263Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCnd).daemonReloadWithBackoff","file.name":"cmd/enroll_cnd.go","file.line":395},"message":"Restarting agent daemon, attempt 0","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-06T07:49:37.265Z","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCnd).Execute","file.name":"cmd/enroll_cnd.go","file.line":205},"message":"Successfully triggered restart on running Elastic Agent.","ecs.version":"1.6.0"}
Successfully enrolled the Elastic Agent.
[====] Done [15s]
Elastic Agent has been successfully installed.
chizzi@ubuntu-24: ~/Downloads/elastic-agent-9.3.4-linux-x86_64$
```

Figure 2: Elastic Agent successfully installed on Ubuntu 24.04 LTS — terminal output confirms enrollment to Fleet Server at localhost:8221, self-signed TLS certificate generated, and agent running as a managed service

The Elastic Agent was then deployed on the Ubuntu 24.04 LTS target from the Downloads directory and enrolled directly as a Fleet Server using the following command:

```
sudo ./elastic-agent install \
  --fleet-server-es=https://localhost:9200 \
  --fleet-server-service-
token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydmVyL3Rva2VuL1RFNzgwNTM1ODc2Mjg6TU9ra
m5wcDIUcmFQMmlKUm01RDhwQQ \
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-es-ca-trusted-
fingerprint=793905feaa8aef75f783870fb4b0707d916dec91499a8142912a632213326d5a \
  --fleet-server-port=8220 \
  --install-servers
```

The terminal output confirmed the full enrollment sequence: self-signed TLS certificate generation, daemon restart, successful enrollment to the Fleet Server at localhost:8221, and final service confirmation — "Elastic Agent has been successfully installed." This mutual TLS binding ensures only authorised endpoints can submit telemetry to the cluster, consistent with Melone's (2021) principle that secure communication channels are foundational to a verifiable monitoring lifecycle.

1.4 System Integration & Log Path Configuration

Within the Fleet UI I added the System Integration to the agent policy. This integration automatically configures the agent to tail the standard Linux authentication and system log paths:

- `/var/log/auth.log*` — authentication events (login attempts, sudo usage, SSH sessions)
- `/var/log/syslog*` — general OS events (kernel messages, service activity, background processes)

These paths are the primary forensic sources for detecting unauthorised access on Linux systems. By mapping them through the Elastic Agent, all log data is automatically normalised to ECS before indexing, enabling consistent KQL queries across any source. As Halder and Ozdemir (2018) argue, the efficacy of an intelligent security system relies on the precision of its input data — ECS field normalisation is the mechanism that delivers this precision across heterogeneous log sources.

1.5 Adversarial Simulation: SSH Brute-Force with Hydra

To generate realistic adversarial telemetry for pipeline validation, I executed a controlled SSH brute-force attack from a Kali Linux node (IP: 172.20.10.5) against the Ubuntu target (IP: 192.168.56.101) using Hydra:

```
sudo hydra -l chizzi -P /usr/share/wordlists/metasploit/unix_passwords.txt -t 1 -W 5 ssh://192.168.56.101
```

The `-t 1` flag (single thread) and `-W 5` flag (5-second wait between attempts) were deliberately chosen to replicate a patient adversary operating at a velocity below naive rate-limiting thresholds. As Stallings and Brown (2018) note, validating a system against known adversarial Tactics, Techniques, and Procedures (TTPs) is essential for security assurance — this simulation provides exactly that adversarial test case. This is also directly relevant to the detection logic evaluation in Part 2, where I analyse how threshold-based rules handle slow-paced attacks.

MITRE ATT&CK Mapping: T1110.001 — Password Guessing (Brute Force sub-technique)

Tactic: Credential Access | Data Source: Authentication Logs | Log Field: `event.outcome = "failure"`

1.6 Multi-Source Log Ingestion

Once the simulation was running, I confirmed that the SIEM was successfully ingesting from three distinct log datasets simultaneously. Multi-source ingestion is critical: a SIEM that only monitors authentication events creates blind spots for system-level lateral movement that appears in syslog but not auth.log. As Stallings and Brown (2018) emphasise, a comprehensive security monitoring system must ingest logs from multiple distinct sources to eliminate visibility gaps in the security architecture.

- system.auth — authentication sub-system events (login success/failure, SSH method, source IP)
- system.syslog — OS-level events (kernel messages, service lifecycle, cron activity)
- elastic_agent.fleet_server — agent health telemetry (policy revision, connectivity status)

Figure 3 shows the consolidated Discover view with real-time ingestion from all three sources confirmed:

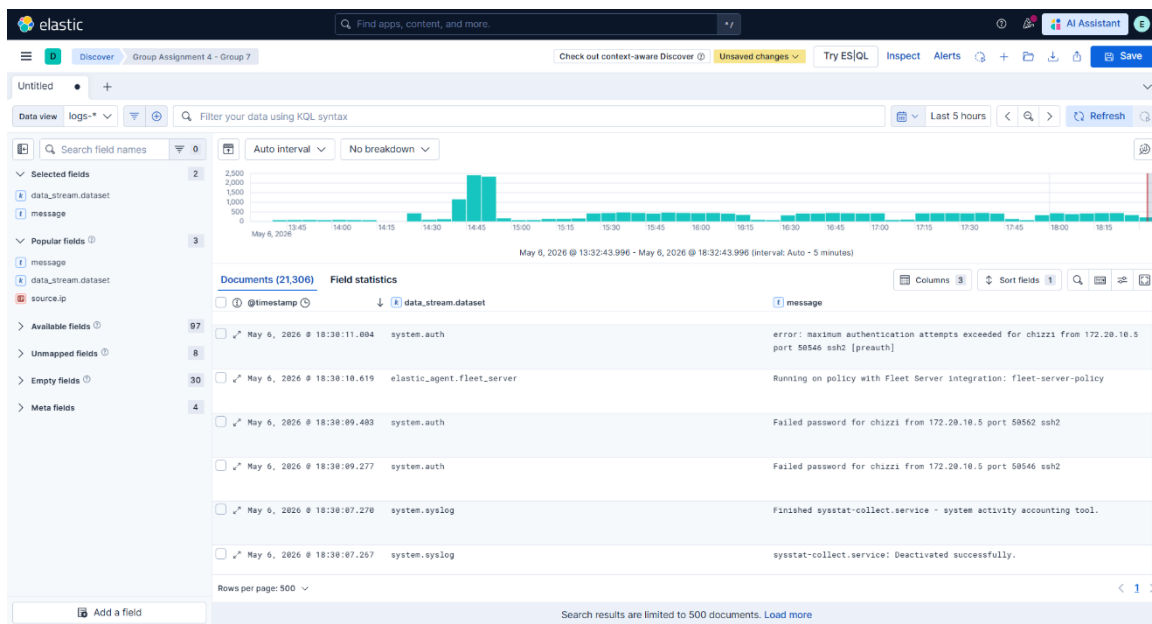


Figure 3: Kibana Discover view — concurrent ingestion from system.auth, system.syslog, and fleet_server datasets confirmed

1.7 Field Parsing & ECS Normalisation

1.7.1 Grok-Based Log Parsing

Raw auth.log entries are unstructured text strings. The SIEM's value is predicated on its ability to tokenise these strings into structured, queryable fields. As Halder and Ozdemir (2018) argue, the utility of a SIEM is predicated on transforming unstructured text into actionable features through parsing — Grok patterns are the mechanism that achieves this. Elastic's ingest pipeline uses Grok to perform this parsing automatically. For example, the raw string:

Failed password for chizzi from 172.20.10.5 port 54321 ssh2

is tokenised into the following ECS-mapped fields:

- user.name: chizzi
- source.ip: 172.20.10.5
- event.outcome: failure

- event.action: ssh_login

Figure 4 shows the JSON document view confirming successful ECS field extraction for a Hydra-generated authentication event:

Document |< < 8 of 500 > >|

Actions: [View single document](#) [View surrounding documents](#)

Table **JSON**

[Copy to clipboard](#)

```
32  "source": {
33    "address": "172.20.10.5",
34    "port": 44608,
35    "ip": "172.20.10.5"
36  },
37  "message": "Failed password for chizzi from 172.20.10.
38             5 port 44608 ssh2",
39  "tags": [
40    "system-auth"
41  ],
42  "input": {
43    "type": "log"
44  },
45  "@timestamp": "2026-05-06T17:19:06.707Z",
46  "system": {
47    "auth": {
48      "ssh": {
49        "method": "password",
50        "event": "Failed"
51      }
52    }
53  },
54  "ecs": {
55    "version": "8.11.0"
56  },
57  "related": {
58    "hosts": [
59      "ubuntu-24"
60    ],
61    "ip": [
62      "172.20.10.5"
63    ],
64    "user": [
65      "chizzi"
66    ]
67  }
```

Figure 4: ECS-mapped JSON document — source.ip (172.20.10.5) and user.name (chizzi) successfully extracted from raw auth.log entry

1.7.2 Indexing Verification

Beyond parsing, I verified that 100% of ingested records were successfully indexed — meaning every field was stored in Elasticsearch's inverted index, enabling sub-second aggregation queries across millions of records. Without complete indexing, statistical analysis such as counting failures per source IP within a time window would return incomplete results.

Figure 5 shows the field statistics panel confirming complete indexing of the `data_stream.dataset` field across all ingested records:

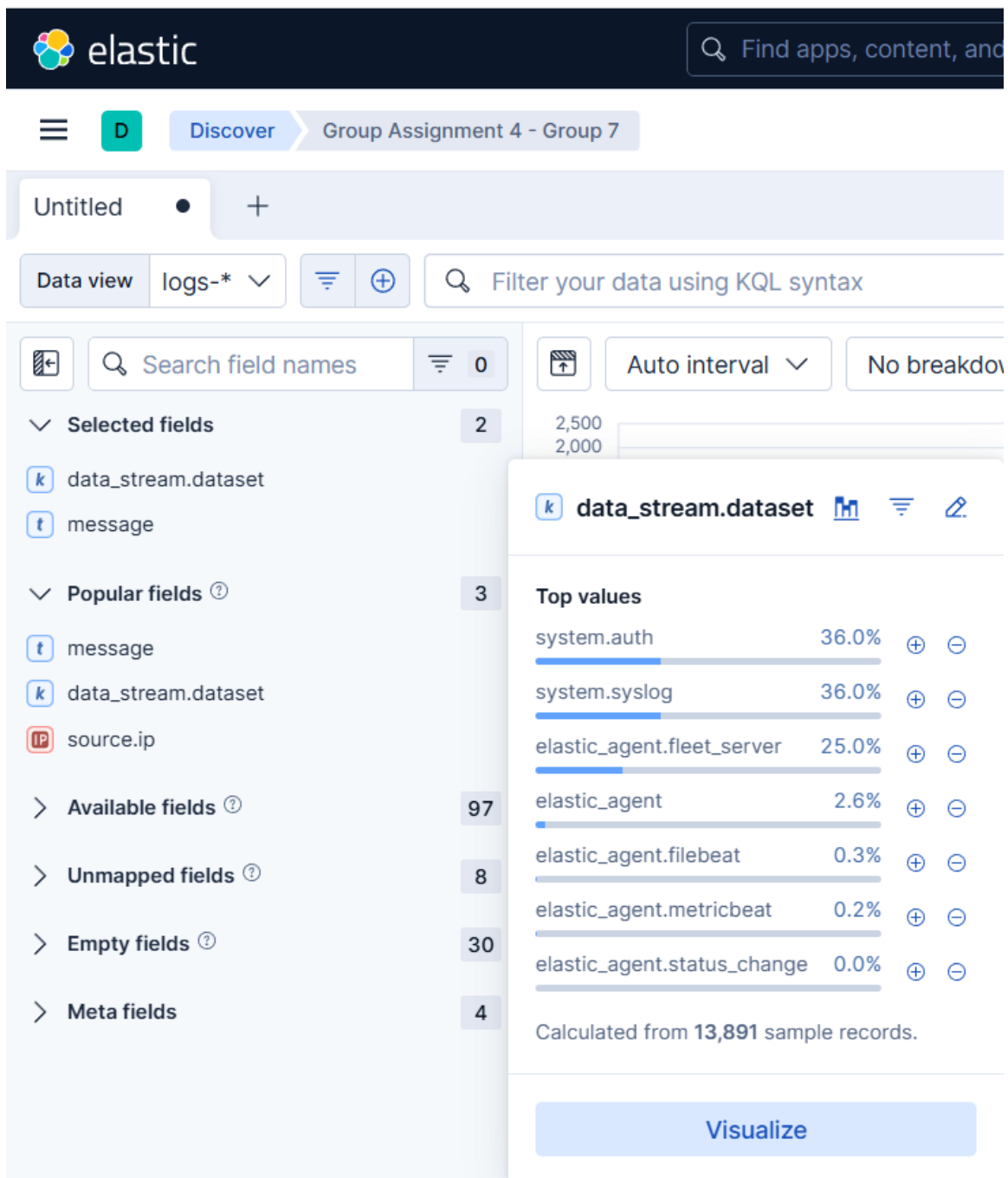


Figure 5: Field statistics panel — 100% indexing confirmed across all ingested records, validating the data lake readiness

1.8 Security Relevance of Collected Data

The three data streams collected serve distinct security functions:

Dataset	Security Function	Example Threat Detection
system.auth	Authentication monitoring	Brute-force detection (T1110), credential stuffing, privilege escalation
system.syslog	OS-level forensics	Post-exploitation activity, lateral movement, persistence mechanisms
fleet_server telemetry	Sensor integrity monitoring	Detection of agent tampering or shut down by an active attacker

Part 2: Detection Engineering & Alert Logic

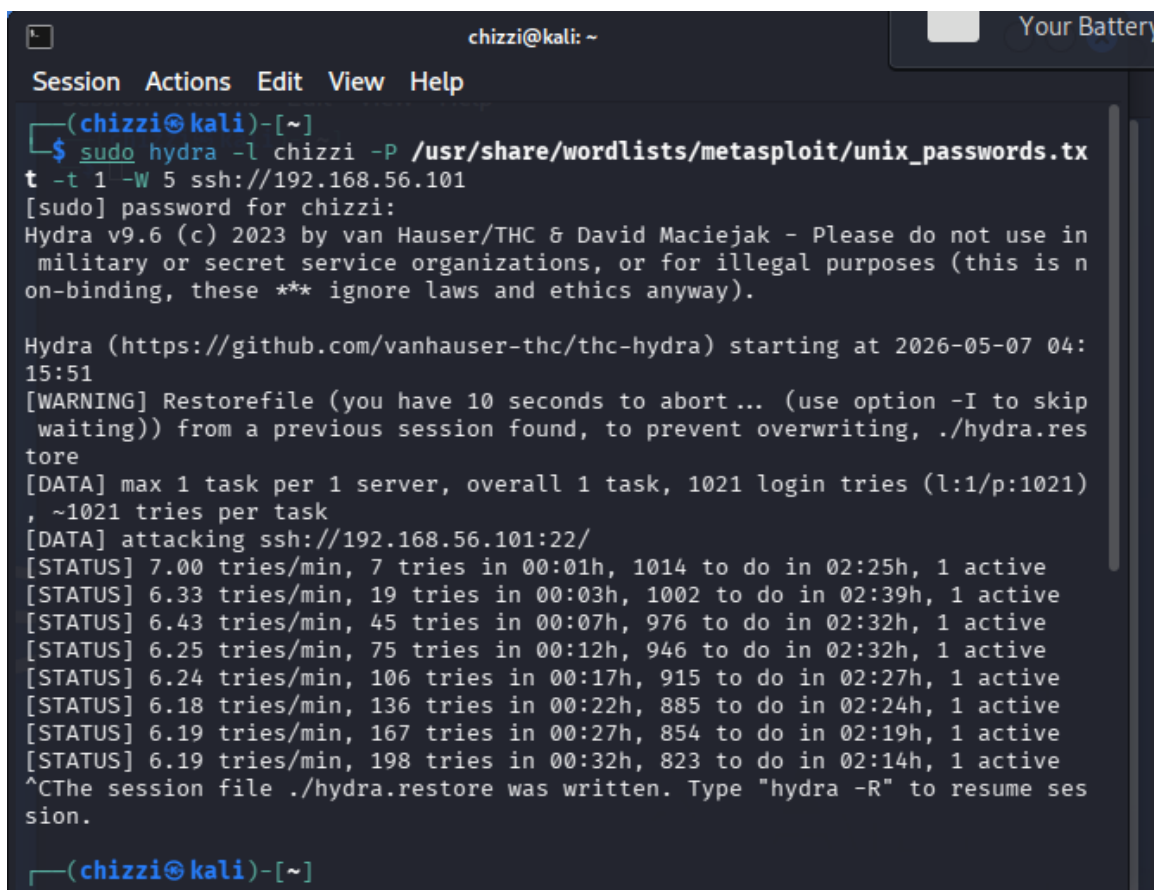
2.1 Use Case Definition

The detection use case I engineered addresses MITRE ATT&CK Technique T1110: Brute Force, specifically the sub-technique T1110.001 (Password Guessing). This is one of the most prevalent initial access vectors against exposed Linux infrastructure. As Stallings and Brown (2018) assert, authentication remains both the first line of defence and the most common point of exploitation — making its robust monitoring a non-negotiable requirement for any SOC.

The goal was to engineer a detection rule that could reliably identify automated brute-force tooling (Hydra) while tolerating the benign authentication failures that occur in any real environment — users mistyping passwords, service account configuration errors, and so on. As Halder and Ozdemir (2018) note, automated attacks generate high-density telemetry that necessitates algorithmic detection rather than manual oversight; the detection rule must therefore distinguish between human fallibility and programmatic exploitation (Hadlington, 2018).

2.2 Adversarial Simulation Detail

Figure 6 shows the Hydra attack in execution, demonstrating the high-velocity authentication failure stream generated against the SSH service:



```
chizzi@kali: ~  
Session Actions Edit View Help  
(chizzi@kali)-[~]  
$ sudo hydra -l chizzi -P /usr/share/wordlists/metasploit/unix_passwords.tx  
t -t 1 -W 5 ssh://192.168.56.101  
[sudo] password for chizzi:  
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in  
military or secret service organizations, or for illegal purposes (this is n  
on-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-07 04:  
15:51  
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip  
waiting)) from a previous session found, to prevent overwriting, ./hydra.res  
tore  
[DATA] max 1 task per 1 server, overall 1 task, 1021 login tries (l:1/p:1021)  
, ~1021 tries per task  
[DATA] attacking ssh://192.168.56.101:22/  
[STATUS] 7.00 tries/min, 7 tries in 00:01h, 1014 to do in 02:25h, 1 active  
[STATUS] 6.33 tries/min, 19 tries in 00:03h, 1002 to do in 02:39h, 1 active  
[STATUS] 6.43 tries/min, 45 tries in 00:07h, 976 to do in 02:32h, 1 active  
[STATUS] 6.25 tries/min, 75 tries in 00:12h, 946 to do in 02:32h, 1 active  
[STATUS] 6.24 tries/min, 106 tries in 00:17h, 915 to do in 02:27h, 1 active  
[STATUS] 6.18 tries/min, 136 tries in 00:22h, 885 to do in 02:24h, 1 active  
[STATUS] 6.19 tries/min, 167 tries in 00:27h, 854 to do in 02:19h, 1 active  
[STATUS] 6.19 tries/min, 198 tries in 00:32h, 823 to do in 02:14h, 1 active  
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume ses  
sion.  
(chizzi@kali)-[~]
```

Figure 6: Hydra SSH brute-force attack in execution — password dictionary attack against `ssh://192.168.56.101` targeting user "chizzi"

Each Hydra attempt generates a corresponding auth.log entry. Figure 7 shows an individual failed authentication log entry as parsed by the SIEM:

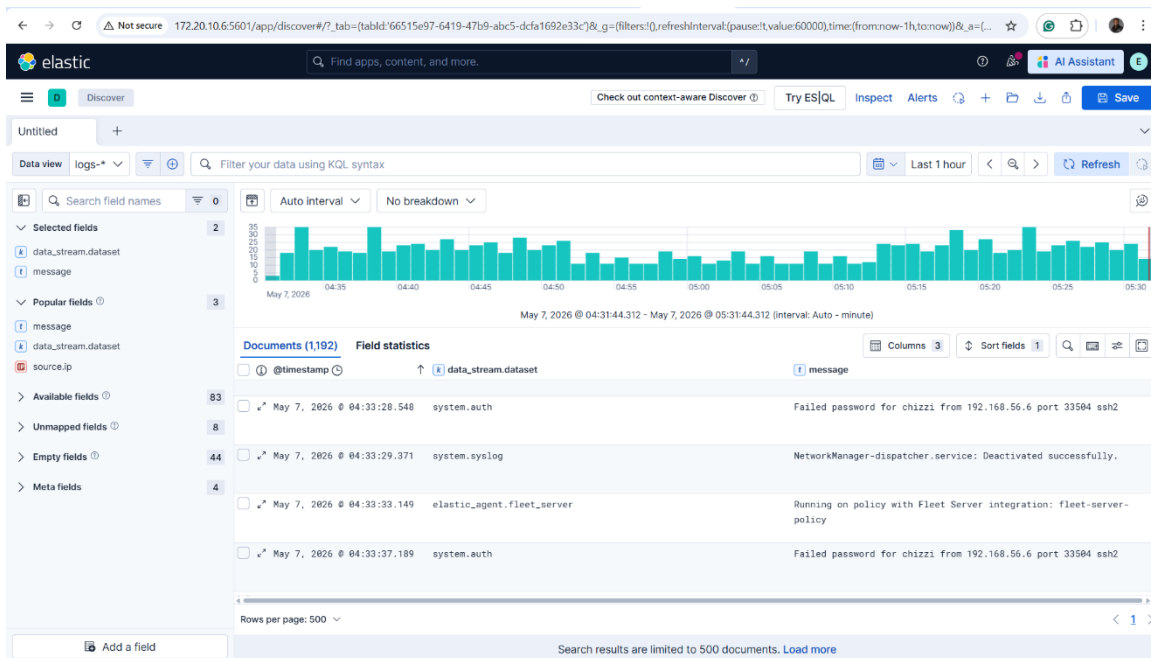


Figure 7: Parsed failed authentication event — SIEM successfully captures event.outcome, source.ip, and user.name from Hydra-generated SSH failure

2.3 KQL Detection Query

The primary detection control is a KQL (Kibana Query Language) filter scoped to the system.auth dataset and filtered to authentication failures only:

```
data_stream.dataset : "system.auth" AND event.outcome : "failure"
```

This query performs two functions simultaneously: it scopes the alert to the correct data stream (eliminating false positives from syslog entries that happen to contain the word "failure"), and it filters to the specific event outcome field that ECS-normalised authentication failures populate. According to Halder and Ozdemir (2018), the efficacy of a detection system relies on the precision of its input filters — this KQL string operationalises that principle by eliminating noise at the query level.

Figure 8 shows the complete detection rule configuration in Kibana, including the KQL filter and metric threshold logic:

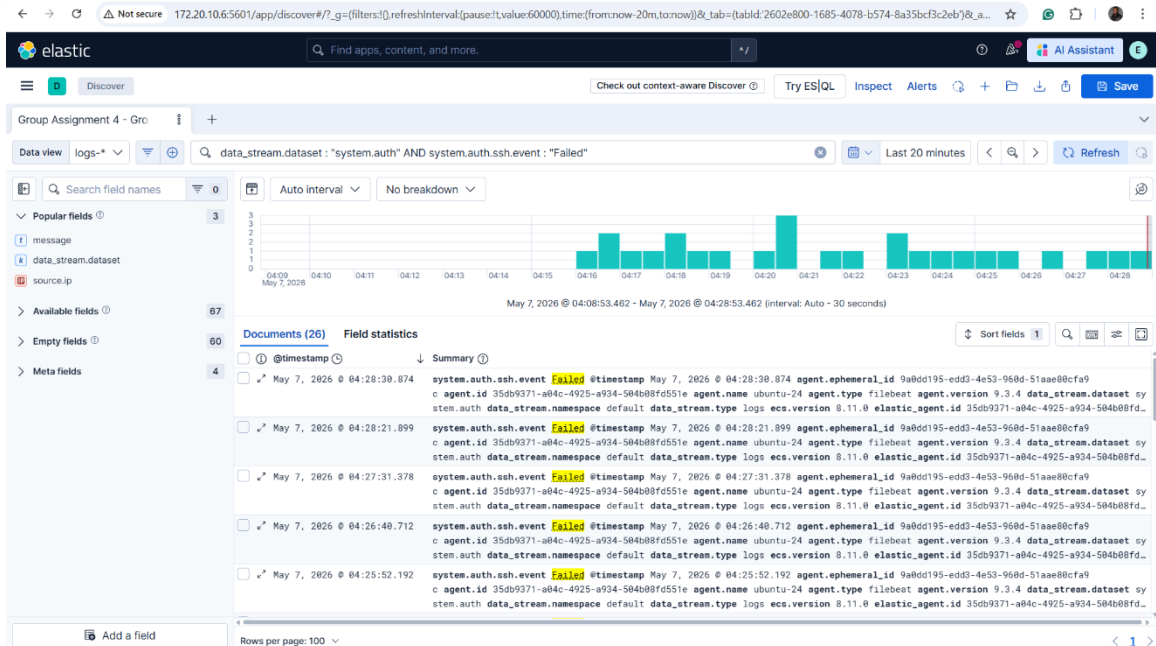


Figure 8: KQL-based detection rule configuration — Metric Threshold rule with source.ip grouping and 10-failure-per-minute threshold

2.4 Threshold Logic & Alert Fatigue Mitigation

The detection rule applies a Metric Threshold: the rule triggers only when more than 10 failed authentication events from a single source.ip are observed within a 60-second window. This design is grounded in the Compliance Budget theory proposed by Beaument, Sasse and Wonham (2008), which argues that security controls must account for the realistic cognitive and behavioural patterns of legitimate users:

- A user who misremembers their password might fail 1-3 times before succeeding or seeking IT support — representing accidental insider behaviour described by Hadlington (2018).
- An automated tool like Hydra, even when rate-limited with `-t 1 -W 5`, will generate failures at a velocity and volume no human user can replicate over a sustained period (Stallings and Brown, 2018).

By establishing a threshold that exceeds human fallibility while remaining sensitive to automated tooling, the rule maintains the Usability standard advocated by Sasse and Flechais (2005): security controls must not become cognitively burdensome to the analyst, but must surface meaningful, actionable signals. The source.ip grouping ensures that the threshold is evaluated per attacker, preventing an adversary from diluting their failure count across multiple targeted usernames.

2.5 Alert Execution & Adversarial Attribution

Figure 9 shows the threshold rule configuration as deployed in Kibana:

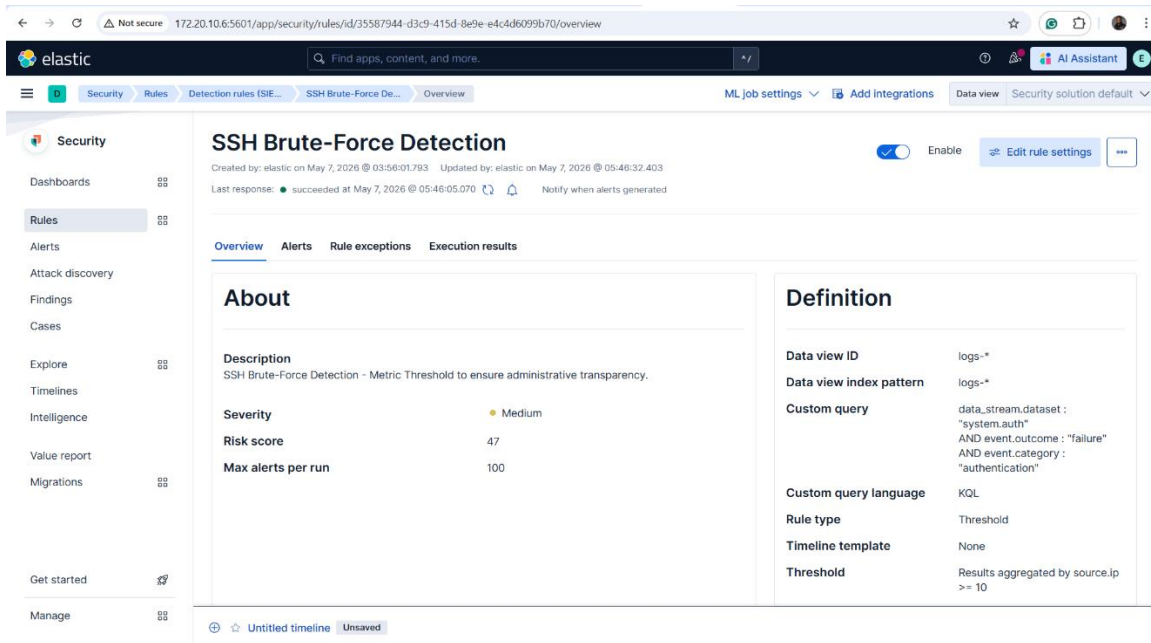


Figure 9: Threshold rule configuration — greater than 10 failures per minute from a unique source.ip triggers a medium-severity alert

Figure 10 shows the alert executing successfully, identifying the Kali Linux adversary node by IP. The source.ip grouping in the alert setup ensures detection is attributed to a specific node rather than aggregated across the network, satisfying the requirement for accountability and attribution in complex systems (Stallings and Brown, 2018). From a socio-technical perspective, this immediate, specific attribution gives the security team the precise evidence required for incident response — moving the organisation from a reactive to a proactive security posture (Ani et al., 2022).

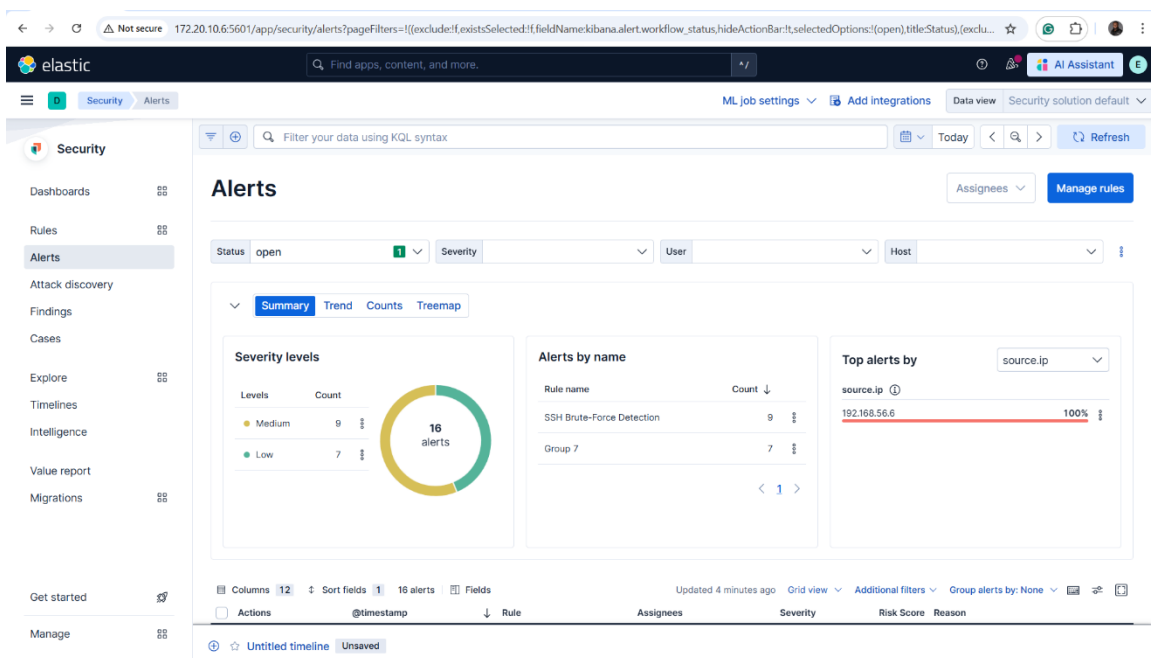


Figure 10: Live alert execution — SIEM successfully identifies the Kali Linux adversary (192.168.56.6) as the source of the brute-force campaign

Attribution result: source.ip = 192.168.56.6 (Kali Linux attacker)
Target: user.name = chizzi | event.outcome = failure | Count > 10/min
Alert severity: MODERATE | Mapped technique: MITRE ATT&CK T1110.001

2.6 Detection Assumptions & Known Limitations

2.6.1 Assumptions

The detection logic operates under three key assumptions that must be documented for any production deployment:

- **Adversarial velocity:** Automated brute-force tools operate at velocities significantly exceeding human input rates, providing a reliable discriminator for threshold-based rules.
- **Static attacker IP:** The source.ip grouping assumes the attacker maintains a consistent IP address throughout the attack window. This is valid for targeted attacks but not for distributed botnets.
- **ECS schema consistency:** The rule depends on Elastic Agent correctly mapping auth.log fields to the event.outcome ECS field. Schema inconsistencies across agent versions could cause missed detections.

2.6.2 Limitations

LIMITATION 1 — Low-and-Slow Evasion: An attacker using -W 5 stays below 12 attempts per minute, approaching the threshold. Reducing the threshold increases false positives — a classic security/usability trade-off (Kainda, Flechais and Roscoe, 2010).

LIMITATION 2 — Distributed Brute Force: A botnet distributing attempts across hundreds of IPs evades source.ip grouping entirely. As Ani et al. (2022) note, attackers adapt to technical controls, requiring progression toward behavioural analytics.

LIMITATION 3 — Contextual Blindness: The rule evaluates each 60-second window in isolation. An attacker making 9 attempts per minute for 60 minutes (540 total) never triggers the alert — a stateful awareness gap identified by Melone (2021).

These limitations inform future development priorities: the next iteration of this detection ruleset should incorporate time-series anomaly detection via Elastic's Machine Learning module, which can detect low-velocity attacks that evade threshold-based rules by modelling normal baseline authentication behaviour.

Part 3: Role-Based Security Dashboards

3.1 Design Philosophy

A single dashboard cannot serve both a Tier-1 SOC analyst triaging alerts in real time and a CISO presenting security posture to a board. As Sasse and Flechais (2005) argue, security tools fail when they do not match users' cognitive needs and workflows — the implication being that a tool designed for one audience actively impedes the other. I therefore designed two distinct dashboards: one optimised for operational triage speed, and one for executive risk comprehension.

The design choices for each dashboard were further informed by Stallings and Brown's (2018) emphasis on continuous monitoring and Hadlington's (2018) human factors perspective: tools must be calibrated to the expertise, cognitive load, and decision authority of their intended audience. A tool that forces an analyst to parse management-level summaries wastes response time; a tool that exposes raw KQL syntax to an executive creates confusion that undermines confidence in the security programme.

3.2 SOC Analyst Dashboard: Real-Time Threat Monitoring

3.2.1 Visualisation 1 — Event Volume Over Time by Dataset

A stacked bar chart provides the primary temporal overview, communicating both total event volume per interval and the proportional contribution of each data source. A SOC analyst opening the dashboard can immediately identify whether current volume falls within normal parameters or represents an anomalous spike requiring investigation.

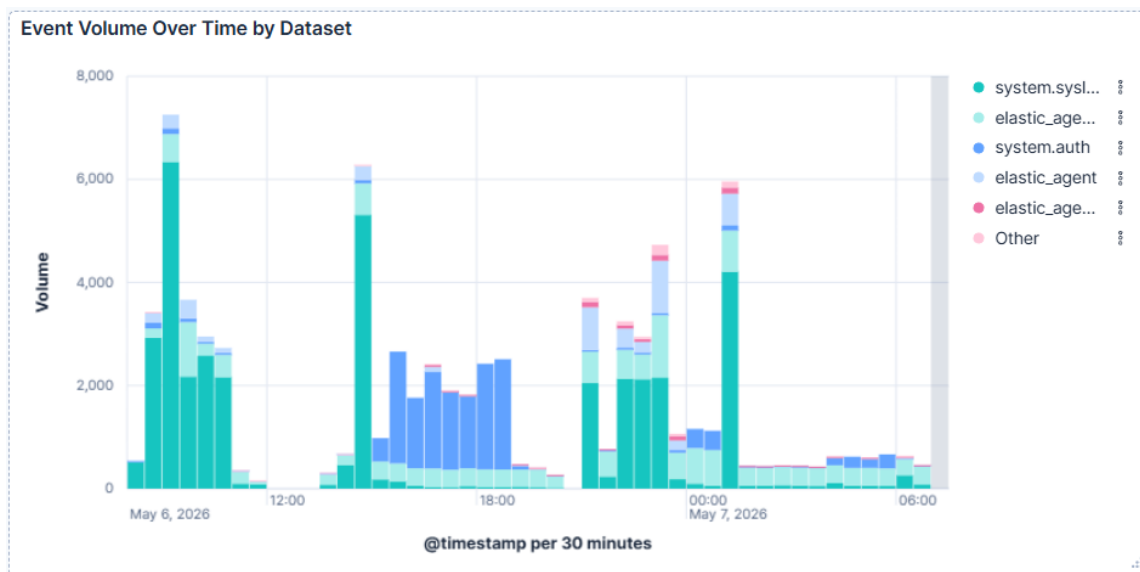


Figure 11: Stacked bar chart — event volume over time by dataset. The cluster at 08:30 on 6 May corresponds to a kernel soft lockup event (CPU#3, 8,812s stuck)

The chart captured a significant event cluster at 08:24 on 6 May 2026, corresponding to a kernel soft lockup. Without the temporal chart, identifying this event would require manual log trawling. The colour-coded dataset breakdown allows the analyst to determine within seconds whether the spike is authentication-related (requiring immediate escalation) or system-level (requiring infrastructure investigation). This

supports the investigation workflow described by Stallings and Brown (2018), who identify the ability to correlate events across multiple data streams as a fundamental requirement of effective security operations.

3.2.2 Visualisation 2 — Failed vs Successful Authentications

A pie chart provides an immediate proportional view of the authentication health of the monitored system. The security question it answers — "what fraction of authentication activity is failing?" — is a proportional question best answered by a part-to-whole chart.

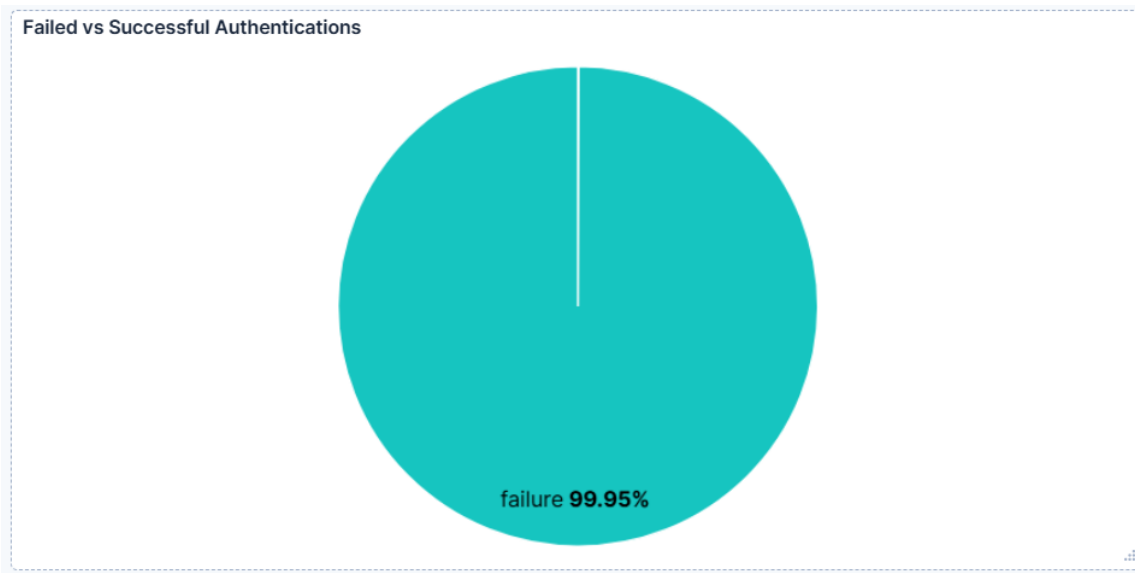


Figure 12: Pie chart — failed vs successful authentication events. 99.95% failure rate is a categorical indicator of active brute-force activity

The dataset revealed a 99.95% authentication failure rate — a finding that would not surface within an operationally viable timeframe through manual log review, but which the dashboard surfaces within seconds of loading. A failure rate of this magnitude is definitively consistent with active automated brute-force activity (MITRE ATT&CK T1110). This aligns with Kainda, Flechais and Roscoe's (2010) argument that usable security tools must reduce cognitive load on the analyst, enabling rapid triage decisions rather than being delayed by information overload.

3.2.3 Visualisation 3 — High-Risk Event Count (KPI Tile)

A single large-number metric tile represents the count of high-risk events, filtered using KQL keyword matching against terms including "failed", "error", "BLOCK", "lockup", and "denied". This supports the pre-attentive processing stage of security triage, consistent with Sasse and Flechais's (2005) user-centred design principle that effective security interfaces should enable rapid, low-effort initial assessment before requiring deeper cognitive engagement.

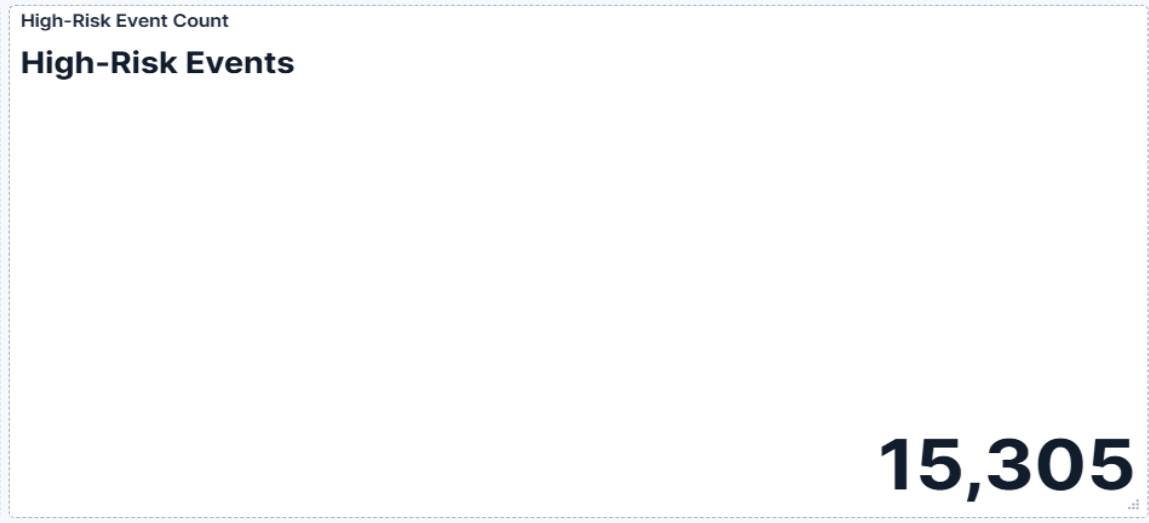


Figure 13: KPI metric tile — 15,305 high-risk events in the monitored period. Establishes baseline for anomaly comparison in subsequent investigations

The metric tile displaying 15,305 high-risk events immediately signals that the environment requires investigation. Critically, this number becomes most useful when compared against a historical baseline — a process I would implement in a production environment by storing daily KPI snapshots and visualising trend deviation.

3.2.4 Visualisation 4 — Top IPs by UFW Firewall Blocks

A ranked vertical bar chart surfaces the top five host IPs associated with UFW firewall BLOCK events. KQL filtering on message: *UFW BLOCK* isolates network-layer blocking events from the broader syslog stream. The descending sort ensures the analyst's attention is drawn to the highest-frequency sources without numerical comparison. As Hadlington (2018) notes, poor situational awareness — including failure to identify which network nodes are generating anomalous traffic — is a primary contributing factor to threats going undetected; this visualisation directly counteracts that failure mode.

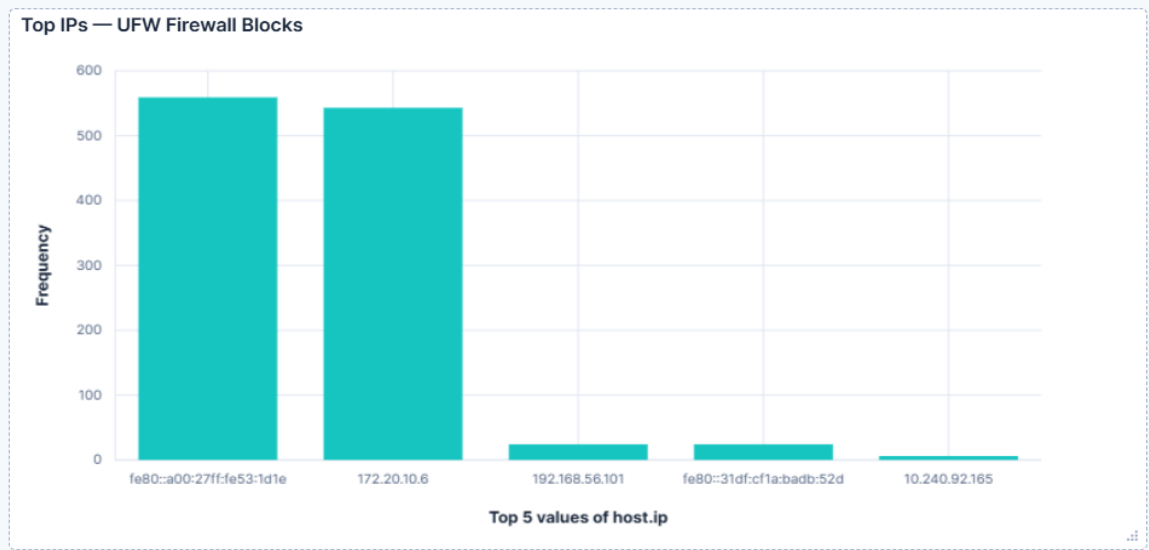


Figure 14: Bar chart — top 5 host IPs by UFW block frequency. Two dominant sources (IPv6 link-local and 172.20.10.6) suggest misconfiguration or persistent external scanning

Note: host.ip was used as a proxy for source IP in UFW block events because the originating IP is embedded in the raw message field rather than a structured ECS field in this dataset. In a production environment I would implement a custom Grok ingest pipeline to extract source.ip from UFW block messages, enabling direct attacker attribution rather than host-level approximation.

3.2.5 Visualisation 5 — Events by Process Name

A pie chart breakdown by process.name surfaces which system processes are generating the greatest share of log activity. This layer of abstraction above raw log data is both accessible to analysts of varying experience levels and actionable within operational timeframes — reflecting Kainda, Flechais and Roscoe's (2010) finding that security interface design must balance information density against cognitive accessibility.

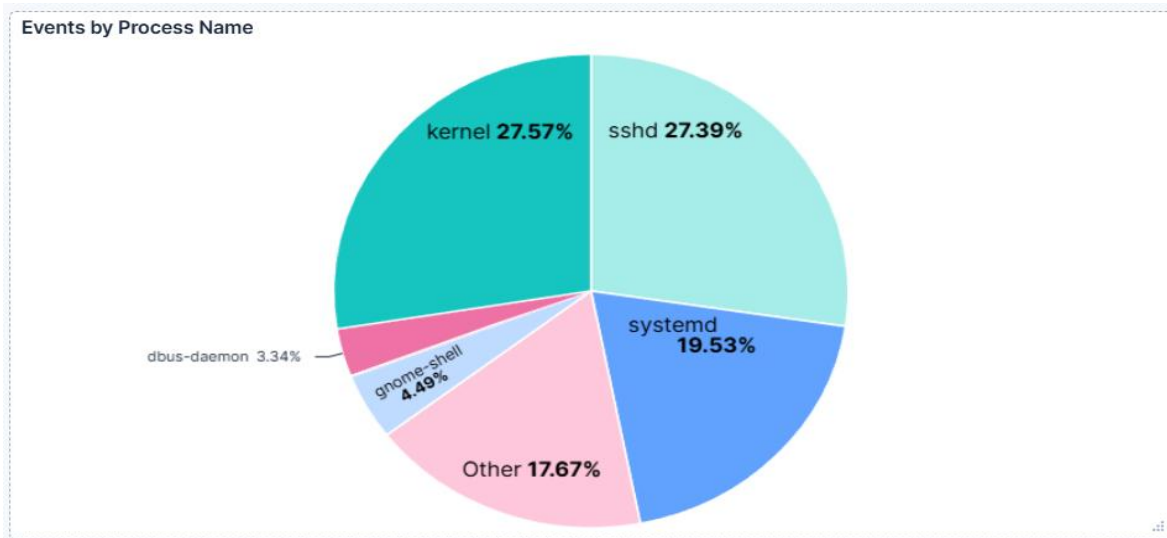


Figure 15: Pie chart — events by process name. sshd (27.39%) and kernel (27.57%) together account for over 55% of log activity

The result — sshd and kernel each accounting for approximately 27% of events — correlates directly with the 99.95% authentication failure rate in Visualisation 2. Combined, these two panels confirm that the SSH daemon is the principal source of anomalous activity, enabling targeted investigation without parsing individual log entries.

3.3 Management Dashboard: Strategic Risk Overview

3.3.1 Visualisation 1 — Security Incident Trend Over Time

An area chart provides a management audience with an intuitive visualisation of security event volume over time. The filled area beneath the trend line creates a visual metaphor for accumulation that is intuitively read by non-technical viewers as "how much risk activity is present" — abstracting away the dataset-level breakdown present in the SOC equivalent. A management reader does not need to distinguish between syslog and authentication events; they need to understand whether the organisation's security burden is stable, increasing, or decreasing (Sasse and Flechais, 2005).

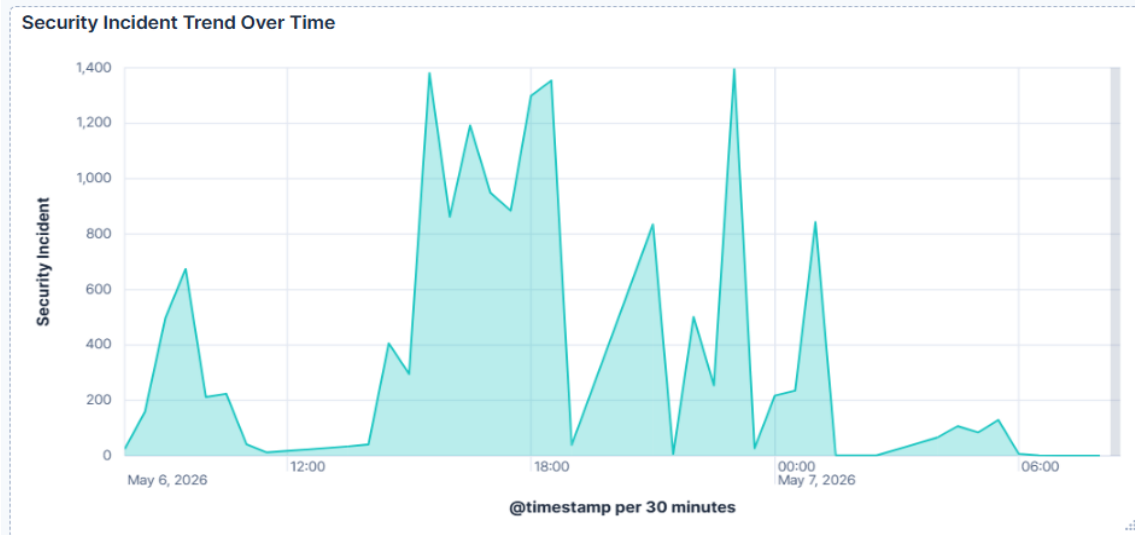


Figure 16: Area chart — security incident trend over time. Visible spikes at 18:00 on 6 May and approaching midnight indicate recurrent high-volume event periods

The visible spikes at 18:00 on 6 May and approaching midnight answer the fundamental management question: "Is our security burden stable or escalating?" This is information a risk officer requires for resource allocation decisions — consistent with Stallings and Brown's (2018) argument that executive visibility of security risk is a governance requirement, not merely an operational convenience.

3.3.2 Visualisation 2 — Incident Source Breakdown

A pie chart presents the proportional split of log events across data sources (system.syslog 48.81%, elastic_agent_fleet_server 23.5%, system.auth 20.08%). This contextualises security information in terms of business impact rather than technical detail — applying the principle articulated by Sasse and Flechais (2005) that security information must be presented in terms comprehensible to non-expert audiences if they are to act upon it effectively.

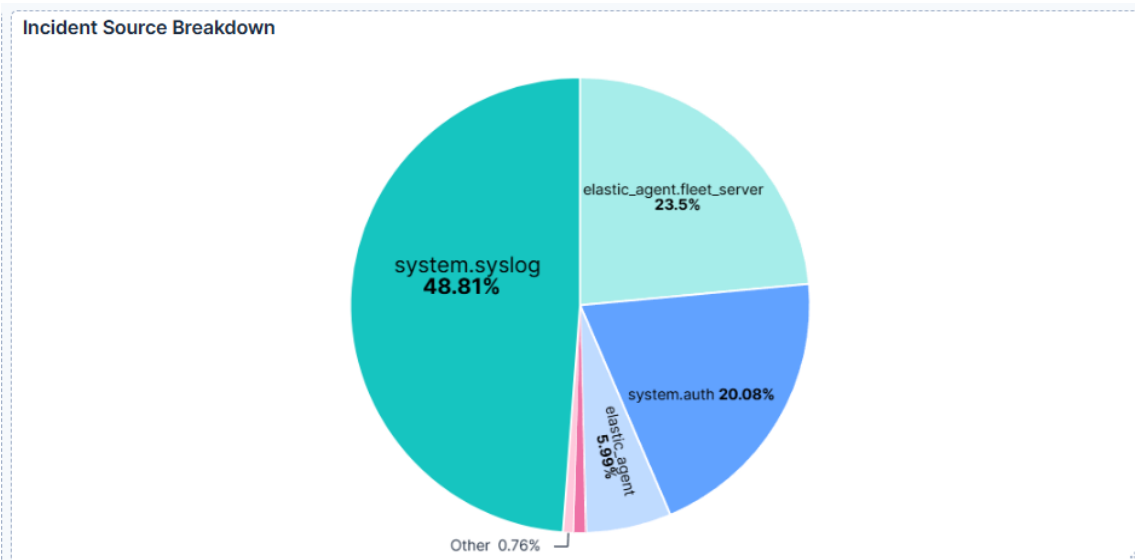


Figure 17: Pie chart — incident source breakdown by dataset. Authentication events (20.08%) cross-referenced with 99.95% failure rate signals active brute-force campaign

A manager who cross-references this chart with the SOC dashboard's 99.95% failure rate can conclude without technical expertise that approximately one-fifth of all security events are attributable to what appears to be an active brute-force campaign against the SSH service. This supports the case for access control hardening — a management-remit decision aligned with the socio-technical governance model advocated by Ani et al. (2022).

3.3.3 Visualisation 3 — Most Frequent Error Sources (Data Table)

A ranked data table presents the top error-generating processes (sshd: 9,044; systemd: 481; kernel: 262). A table was chosen over a bar chart for the management context because it presents precise, named information in a format familiar from business reporting. As Sasse and Flechais (2005) argue, security information must be presented in terms of business impact rather than technical detail for non-expert audiences to act upon it effectively — a table satisfies this requirement more directly than a chart requiring visual interpretation.

Most Frequent Error Sources	
Sources Of Error	Frequency
sshd	9,044
systemd	481
kernel	262
gnome-shell	195
gsd-media-keys	60
Other	494

Figure 18: Data table — most frequent error sources. sshd with 9,044 errors (18x the second-ranked process) provides clear evidence for SSH service review prioritisation

The dominance of sshd with 9,044 errors — more than eighteen times the second-ranked process — translates directly into a management-level action: the SSH service is generating a disproportionate security burden and must be reviewed as a priority. This addresses Hadlington's (2018) observation that poor executive decisions arise from a lack of contextualised security information.

3.3.4 Visualisation 4 — System Health Gauge

A linear gauge displaying the system error load (14,730 errors) provides an executive summary of system health in a single visual element. The gauge metaphor mirrors everyday analogues — fuel gauges, battery indicators, progress bars — that non-technical audiences interpret instinctively. By mapping error volume to a positional indicator, the visualisation translates a raw number into a directional judgement, consistent with Stallings and Brown's (2018) argument that executive visibility of security risk is a governance requirement.



Figure 19: Linear gauge — system health indicator at 14,730 errors. Near-maximum position indicates high error load requiring investigation without requiring technical interpretation

In a production deployment I would configure colour-banded thresholds (green/amber/red) in Elastic's gauge settings, enabling the gauge to function as a simple RAG status indicator. A CISO could then answer the question "how is our security health?" before a board meeting without consulting an analyst.

3.4 Design Differentiation Summary

Design Element	SOC Analyst Dashboard	Management Dashboard
Primary question answered	What is happening right now?	Is our security posture stable?
Time granularity	Minute-by-minute (operational)	Hourly/daily trend (strategic)
Data source breakdown	Colour-coded by dataset	Consolidated single-series
Technical detail	ECS field names, KQL filters visible	Plain process names, no query syntax
Primary chart type	Stacked bar (multi-series temporal)	Area chart (narrative accumulation)
Status indicator	KPI tile (precise count)	Gauge (directional health indicator)

Part 4: Critical Reflection

4.1 Technical Strengths

The most significant technical achievement of this project was the successful end-to-end validation of the detection pipeline: from raw SSH authentication failure in `/var/log/auth.log`, through ECS field extraction, to a live alert attributing the attack to a specific source IP. As Melone (2021) argues, a secure system must not only protect and detect but also provide a clear audit trail for recovery — each stage of this chain was validated with screenshots, creating precisely that auditable evidence trail.

The threshold-based detection rule performed as designed: it correctly identified the Hydra attack while the deliberate -W 5 inter-attempt delay was in place, demonstrating that the 10-per-minute threshold remained sensitive enough to catch rate-limited automated tooling. The `source.ip` grouping provided the forensic attribution required for a real incident response — an analyst could immediately escalate to a network team to initiate IP-level blocking or geolocation analysis (Stallings and Brown, 2018).

4.2 Detection Threshold Trade-offs

The central design tension in threshold-based detection is the inverse relationship between sensitivity and specificity. Lowering the threshold to catch slower attacks increases the false positive rate, generating alert fatigue — the failure mode identified by Beaument, Sasse and Wonham (2008) as one of the primary mechanisms by which security controls lose their operational effectiveness over time.

In this project I observed the practical manifestation of this trade-off: the Hydra -W 5 flag pushed the attack velocity close to the threshold boundary. An attacker aware of the 10-per-minute threshold could calibrate their tool to operate at exactly 9 attempts per minute indefinitely, making 540 attempts per hour without triggering a single alert. This is the "low-and-slow" evasion technique that threshold-based rules cannot address — a limitation that reflects the trade-off between security and usability identified by Kainda, Flechais and Roscoe (2010).

The mitigation for this limitation is time-series anomaly detection using machine learning. Rather than applying a static threshold, an ML-based rule models each source IP's historical authentication behaviour and alerts when the current rate deviates significantly from that baseline — the approach described by Halder and Ozdemir (2018) as essential for detecting attacks that operate below fixed thresholds.

4.3 Distributed Attack Limitations

The `source.ip` grouping logic assumes a single attacking IP. In a realistic botnet-based brute-force campaign, the attacker may distribute attempts across thousands of compromised nodes, each contributing only 1-2 attempts per hour — well below any per-IP threshold. The SIEM as configured would generate no alert despite a sustained credential attack at scale. As Ani et al. (2022) note in their socio-technical modelling, attackers frequently adapt to technical controls, requiring security systems to progress toward more complex behavioural analytics rather than relying on static rules.

Addressing this requires aggregating failures by `user.name` rather than (or in addition to) `source.ip`: if the account "chizzi" accumulates 1,000 failed login attempts in an hour from 500 different IPs, a user-centric detection rule would fire even though no individual IP exceeds the per-IP threshold. A production deployment of this ruleset would implement both detection strategies in parallel, consistent with the defence-in-depth model advocated by Stallings and Brown (2018).

4.4 Governance, Privacy & UK GDPR Considerations

Any production SIEM deployment in a UK organisation processes personal data: `user.name` and `source.ip` are both personal data under the UK GDPR (post-Brexit retained regulation under the Data Protection Act 2018). A deployment of this nature would require:

- A Data Protection Impact Assessment (DPIA) prior to go-live, assessing the necessity and proportionality of collecting behavioural telemetry.
- A documented lawful basis for processing — most likely legitimate interests or legal obligation for security monitoring purposes.
- A defined data retention policy governing how long authentication logs and alert records are held before automated deletion.
- Role-based access controls ensuring that raw telemetry (including `user.name` fields) is accessible only to authorised security personnel, not exposed through the management dashboard.

The continuous monitoring architecture also raises workplace governance considerations. A SIEM that logs every user authentication event creates a comprehensive behavioural record of employee activity. As Hadlington (2018) notes, disproportionate or opaque monitoring can foster distrust and circumvention — staff may disable logging or use unmonitored channels, creating the very blind spots the SIEM was designed to address. Zimmermann et al. (2024) similarly argue that approaches which treat employees as surveillance targets rather than security partners are counterproductive, and that a transparent, scoped, and role-controlled SIEM represents a more sustainable governance model.

A transparent monitoring policy, communicated clearly to staff and scoped strictly to security purposes, is therefore not only a compliance requirement but a practical security requirement. Following Ani et al.'s (2022) socio-technical perspective, SIEM governance must balance intrusive telemetry against users' privacy rights, ensuring monitoring remains transparent and ethically aligned rather than purely focused on technical surveillance.

4.5 Future Development Roadmap

Priority	Enhancement	Rationale
High	ML anomaly detection (Elastic ML module)	Addresses low-and-slow evasion
High	User.name-grouped distributed attack rule	Addresses botnet evasion
Medium	UFW Grok ingest pipeline	Structured source.ip from firewall logs
Medium	SOAR integration (auto-block via iptables)	Automated response to confirmed attacks
Low	OT/ICS log source integration (Nozomi)	Extend to industrial environments (CGAI)

References

- Ani, U.D., Watson, J.M., Tuptuk, N., Hailes, S. and Jawar, A. (2022) Socio-technical security modelling: analysis of state-of-the-art, application, and maturity in critical industrial infrastructure environments/domains. Keele University. Available at: <https://keele-repository.worktribe.com/output/506855>
- Beautement, A., Sasse, M.A. and Wonham, M. (2008) 'The compliance budget: managing security behaviour in organisations', in Proceedings of the 2008 Workshop on New Security Paradigms. New York: ACM Press, pp. 47-58. Available at: <https://www.nspw.org/papers/2008/nspw2008-beautement.pdf>
- Hadlington, L. (2018) 'The "Human Factor" in Cybersecurity: Exploring the Accidental Insider', in Psychological and Behavioral Examinations in Cyber Security. Hershey, PA: IGI Global. Available at: <https://irep.ntu.ac.uk/id/eprint/37590/>
- Halder, S. and Ozdemir, S. (2018) Hands-on machine learning for cybersecurity: safeguard your system by making your machines intelligent using the Python ecosystem. Birmingham: Packt Publishing.
- Kainda, R., Flechais, I. and Roscoe, A.W. (2010) 'Security and usability: analysis and evaluation', in 2010 International Conference on Availability, Reliability and Security. Available at: <https://ieeexplore.ieee.org/document/5438081>
- Melone, M. (2021) Designing Secure Systems. Sebastopol, CA: O'Reilly Media.
- MITRE Corporation (2023) ATT&CK Technique T1110 — Brute Force. Available at: <https://attack.mitre.org/techniques/T1110/>
- NCSC (2022) Introduction to logging for security purposes. National Cyber Security Centre. Available at: <https://www.ncsc.gov.uk/guidance/introduction-to-logging-for-security-purposes>
- Sasse, M.A. and Flechais, I. (2005) 'Usable security: why do we need it? How do we get it?', in Security and Usability: Designing Secure Systems that People Can Use. Sebastopol, CA: O'Reilly Media. Available at: <https://discovery.ucl.ac.uk/id/eprint/20345/>
- Stallings, W. and Brown, L. (2018) Computer security: principles and practice. 4th edn. Harlow: Pearson.
- Zimmermann, V., Schoni, L., Schaltegger, T., Ambuehl, B., Knieps, M. and Ebert, N. (2024) 'Human-centered cybersecurity revisited: from enemies to partners', Communications of the ACM, 67(11), pp. 72-81. Available at: <https://dl.acm.org/doi/10.1145/3665665>